Machine Learning
Coursework Two Report
Aria Gharachorlou
GitHub Repository : `https://github.com/0xbrimp/MLCoursework_2`
Please note : all images are available in full resolution in the GitHub, with references of which images to check in the ReadMe

1 Motivation Behind the Problem

Active Learning *(AL)* addresses the challenge of reducing annotation cost by selecting the most informative examples from a large pool of unlabelled data. This is especially true within **domains** that have expensive processing requirements or entail expert knowledge for labelling, such as **medical imaging**, **scientific analysis** and **industrial inspection / analysis**. The central idea is that the model can achieve *high performance** with fewer labelled samples if it chooses which samples to label strategically. [1]
However, modern deep **AL** methods suffer from poor performance when the labelling budget is extremely small. As detailed in *Hacohen et al. (2022)* a *"Cold Start"* problem arises because uncertainty-based strategies, often dominant in the AL literature, depend on a well-trained model to estimate uncertainty with high reliability. **As a result**, with a handful of labelled samples, these estimates often collapse, causing **AL** to perform no better than a *random sampling process* [confirmed by my later testing][2.2] and sometimes worse.[1],[2]

Hacohen's paper also identifies that the low budget regime behaves *"Fundamentally Differently"* from the high-budget regime. With the **theoretical analysis** revealing a *phase transition*, when labels are scarce, sampling typical *high-density & representative* examples is optimal. On the contrary, when labels are plentiful, sampling *atypical* or uncertain examples becomes beneficial. This insight motivates the development of **TypiClust**, a method explicitly designed for the low-budget regime.
Its significance (TypiClust's) extends broadly across machine learning (ML), enabling effective learning in settings where annotation is the primary bottleneck for data pieces.

2 Adopted Methodology

The methodology implemented in this coursework, follows the proposed by *Hacohen et al. (2022)* [1], adapted to the **CIFAR-10** dataset and reproduced in full within the accompanying **GitHub Repository**. The pipeline consists of three stages, learning a representation space using self-supervised learning,**under a strict label budget**, here using $[10, 20, 40, 80]$, computing typicality and clustering to ensure representative and diverse sample selection, and finally selecting informative samples under a strict label budget. The *outputs* directly feed into the quantitative and qualitative results presented in later sections.
Because the low-budget system provides very few labelled examples, the first stage of the pipeline trains *without any labels.*
A ResNet-18 encoder is trained using the **NT-Xent Contrastive Loss**, where each image is augmented twice, to create two correlated views*. The encoder is optimised to maximise agreement between these two views, while pushing representations of different images.
This provides semantically meaningful depth to the feature space, ensuring suitability for typicality estimation.
The convergence behaviour of the SSL stage is shown [2.1], where the **NT-Xent Loss** decreases steadily over **500 epochs** : confirming stable representation learning. *500 Epochs were selected to match the conditions of the paper* [1]

My implementation involved **500 epochs** over CIFAR's 50,000 images, with a **batch-size** of 128. The total **SSL Iterations** performed would be $\frac{50{,}000}{128} \cdot 500 \approx 195{,}000$ SSL Iterations[1.3]. This is less than the paper's 400,000, due to time constraints and limited hardware.
After computing typicality, TypiClust selects **one representative sample per cluster** to ensure broad coverage of the feature space before allocating the remaining budget to the **highest-typicality** points. For its iterations, the algorithm proceeds **in rounds**, selecting a fixed number of samples per iteration until the budget is exhausted.
Once SSL training completed, the encoder is used to extract feature vectors *for all CIFAR-10* training images. This step is **essential** as **TypiClust** relies on the geometry of the learned feature space rather than on model uncertainty, which is seen to be unreliable in a low-budget scenario.[1],[2.2]
Informative data samples located within the dense data manifold. These are the **typical examples**, which are computed using *k-nearest neighbours.*
1) For each feature vector, the average squared distance to its neighbours is computed, $\alpha = \frac{1}{k}(\|f_i - f_{i_j}\|)^2$
2) The typicality is calculated using the formula $T_i = \frac{1}{\alpha_i + \epsilon}$ The addition of the typicality estimation helps prioritise representative examples, addressing the aforementioned *"cold start"* problem [1],[2]
Overall, **my methodology** implements the **fully-supervised & self-supervised embedding**, along with **semi-supervised** settings. In the fully-supervised framework implemented, only selected labelled examples are used to train a classifier. This allows for comparison between **TypiClust** and **uncertainty-based baselines**, such as *Margin, Entropy, BALD, DBAL*[2.3.1].
Analysis showed that TypiClust maintained stable performance. The SimCLR encoder to compute typicality had a cumulative effect[2.3.2] where performance would increase drastically across lower budgets. These findings are concordant with the original paper's findings.[1]

3 Statistical Analysis

A **paired t-test** was performed between TypiClust and Random Selection ^{Table5} for each budget using the **same five seeds** to ensure **sample dependence**.^{Table2} At budgets 40 and 80, the improvement of TypiClust over Random was statistically insignificant ($p < 0.05$), confirming that the method provides a measurable advantage once the labelled data reaches a minimal size.
At budgets 10 and 20, no significant difference was observed due to the high variance and inherent difficulty of the cold-start problem.[2.2] These findings are concordant with Hacohen's findings[1] in his paper. The total runtime of the **unmodified** algorithm elapsed **3 hours and 5 minutes** to create results for the 500 epochs processed.

While K-means provides a computationally efficient clustering mechanism, it assumes **spherical clusters of similar density**. In practice it's not warranted that the SSL feature space will exhibit such, and may have imbalanced or highly irregular cluster shapes. This behaviour motivated me to modify **the algorithm**[1.1].
Initially, Density-based clustering methods inspired me to use a **DBSCAN substitute** for K-means.
However, DBSCAN **proved impractical** as its performance deteriorated dramatically in high-dimensional spaces and had quadratic computational growth $O(n^2)$ when applied to **512-dimensional SSL embeddings**. To remedy this, I opted to adopt a **HDBSCAN** algorithm, a similar method, more robust to parameter choices and better suited to the irregular high-dimensional manifolds.^{Table3}
The **comparative performance** of the **three TypiClust Frameworks** is summarised with Figures **4.1-4.3**. **All** three frameworks performed **better** than the **random baseline** *beyond* a budget of 40. Generally, the **supervised** framework had dominated over the other frameworks, **concordant with Hacohen's results** [1],[4,1],[4.2]
The distribution of selected indices for a sample budget (*B=10*) shows that each framework **samples distinct regions** of the feature space, reflecting differences in underlying representations.[4.3],[6]

4 Algorithm Modification and Evaluation

In conjunction to the HDBSCAN algorithm implemented, a diversity penalty parameter λ was incorporated to regular the **trade-off** between **typicality** and **cluster coverage**. Results varied across budgets, though a **general increase** in performance (accuracy) was seen at lower budgets of 10 and 20, [3.1],[3.2] but a decrease in performance comparatively to the other two strategies at higher budgets of 40 and 80. ^{Table4}

For the **50,000 CIFAR-10** embeddings taken into consideration, **a reduced amount** compared to other sources[1] due to time concerns, approximately $O(n^2) = 50,000^2$ **distance computations** had to be taken. To mitigate this, the dataset was resampled to **20,000** points, reducing the computational load by 84%, with this reduction, clustering took **32 minutes**, and the training of the TypiClust algorithm took a simiarly paced **3 hours and 11 minutes**.

To **further accelerate** clustering, **PCA** was applied to **reduce** the 512-dimensional SSL Embeddings to **50 principal components**, this produced a representation approx. 10x smaller, whilst preserving neighbourhood structure. HDBSCAN processed the reduced space significantly (*circa. 1.23 times*) **faster**, enabling for practical experimentation. The selected points used in clustering lie in dense, central regions of the manifold. [3.4]

A paired t-test was undertaken similarly to before, between the **Random Baseline** and across five seeds. ^{Table5}, K-means is seen to be unstable (p-value wise) at mid-range budgets, and strong at extremes, **a stark contrast** to the modified HDBSCAN variant, performing better at lower budgets, and comparatively worse at higher budgets. ^{Table5} [3.1]
In **relation to Hacohen's paper**[1], the high variance and weak separability of the feature space at *low budgets* explain why the t-test shows **no statistically significant difference** between **TypiClust** and **Random Selection**.

This directly reflects on the 'cold-start' behaviour described by Hacohen et al.[1], where typicality signals are too unstable to provide an advantage when a handful of labels (low-budget) is available. As the budget increases, the t-test performed[^{Table5}] shows meaningful improvements for TypiClust, **supporting the paper's claim** that typicality-based sampling become increasingly **once enough labelled examples** stabilise the representation space, this trend **persisted across all three frameworks**[3.1],[4.2]

5 Conclusion

The unmodified TypiClust algorithm designed, successfully replicated the result's Hacohen et al. (2022) reported using the CIFAR-10 dataset under strict label budgets.[1] The findings align with the phase-transition theory proposed, highlighting the importance of typicality and diversity in a low-budget regime.[2.4],[2.5]
The instability of K-means in high-dimensional feature spaces ultimately inspired the adoption of the HDBSCAN variant, applying PCA for dimensionality reduction, and subsampling the dataset to reduce computational overhead. The modified algorithm achieved more stable cluster structures and improved performance at lower budgets. [3.2]
Although the improvement was **not statistically significant** across all budgets, the modification provides a more robust foundation for typicality-based active learning. **Future** work could explore **hybrid clustering** strategies or **adaptive density thresholds** [2],^{Table4} to further enhance performance.

6 Appendix

Please note : Figures and tables beginning with a **2** are related to the **Unmodified Algorithm**. Likewise, Figures and tables beginning with a **3** are related to the **Modified Algorithm**.

Additional Disclaimer : Although **not explicitly used in Hacohen's paper**, I opted to use an **unsupervised learning** framework in addition to the SSL and Supervised-Learning used in **Hacohen's paper**, to observe the differences in performance.

**High Performance* : Reduced processing requirements for computation of an algorithm

***Correlated Views* : Data pieces derived from the same source (input)

[1] : `https://proceedings.mlr.press/v162/hacohen22a.html` **: Hacohen et al. (2022)** - Active Learning on a Budget : Opposite Strategies Suit High and Low Budgets

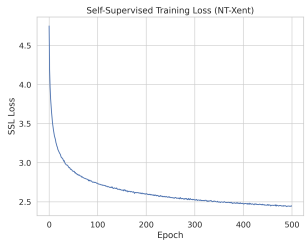
[1.1] : **Campello et al., (2013)** Density-based clustering methods

[1.2] : `https://www.cs.toronto.edu/~kriz/cifar.html` **: CIFAR-10 & CIFAR-100 Datasets**

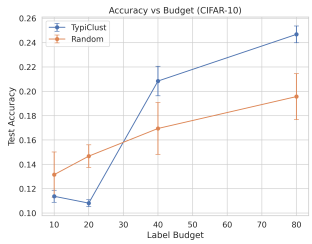
[2] : **Attenberg, J., & Provost, F.(2010).** Why label when you can search? NIPS Workshop on Active Learning.

Budget	TypiClust (mean ± std)	Random (mean ± std)
10	0.113 ± 0.005	0.131 ± 0.018
20	0.108 ± 0.003	0.147 ± 0.010
40	0.208 ± 0.013	0.169 ± 0.020
80	0.247 ± 0.006	0.195 ± 0.019

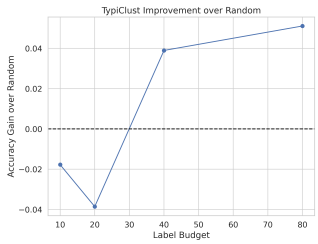
Table 1: Mean test accuracy across 5 seeds for TypiClust and Random selection on CIFAR-10.



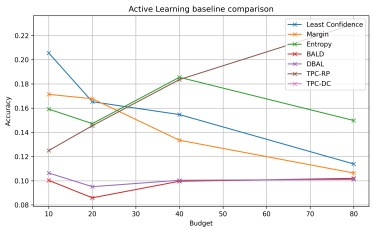
(a) Figure 2.1: Self-Supervised Learning (SSL) loss curve over 500 epochs, using a contrastive NT-Xent algorithm



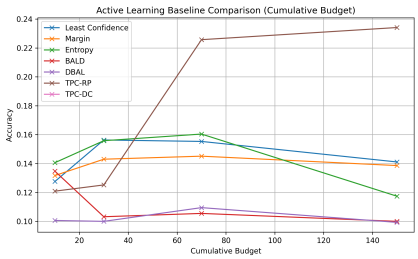
(b) Figure 2.2: Visualises the performance of TypiClust and a Random method, for comparison across different budgets



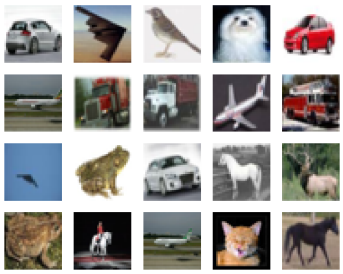
(c) Figure 2.3: Highlights the total gain the algorithm over 500 epochs received and the relative improvement over various budgets



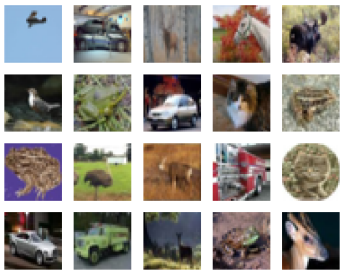
(a) Figure 2.3.1: Comparison of different frameworks against the TPC_{rp} and TPC_{dc} algorithms, for *budget* and *accuracy*



(b) Figure 2.3.2: Shows the cumulative budget against different models - TPC_{dp} omitted



(a) Figure 2.4: Results generated from TypiClust's selected samples algorithm



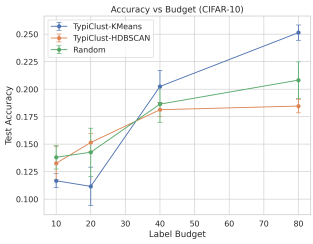
(b) Figure 2.5: Results generated from a Random Selection sampling algorithm

Method	**Budget**	**Seed**	**Accuracy**	**Runtime (s)**
TypiClust	10	0	0.1127	4.536
Random	10	0	0.1290	4.669
TypiClust	20	0	0.1097	4.938
Random	20	0	0.1336	4.978
TypiClust	40	0	0.2108	6.318
Random	40	0	0.1340	5.698
TypiClust	80	0	0.2417	6.597
Random	80	0	0.1963	6.629
TypiClust	10	1	0.1088	5.441
Random	10	1	0.1562	4.686
TypiClust	20	1	0.1042	4.891
Random	20	1	0.1532	4.950
TypiClust	40	1	0.2202	5.625
Random	40	1	0.1791	5.677
TypiClust	80	1	0.2468	6.623
Random	80	1	0.1745	6.510
TypiClust	10	2	0.1188	4.648
Random	10	2	0.1339	5.383
TypiClust	20	2	0.1117	5.002
Random	20	2	0.1538	5.012
TypiClust	40	2	0.2170	5.649
Random	40	2	0.1896	6.299
TypiClust	80	2	0.2402	6.628
Random	80	2	0.2032	6.590
TypiClust	10	3	0.1094	4.671
Random	10	3	0.1041	5.726
TypiClust	20	3	0.1068	5.228
Random	20	3	0.1401	5.145
TypiClust	40	3	0.1896	5.703
Random	40	3	0.1766	5.818
TypiClust	80	3	0.2475	7.648
Random	80	3	0.1817	6.755
TypiClust	10	4	0.1192	4.695
Random	10	4	0.1345	4.809
TypiClust	20	4	0.1084	5.953
Random	20	4	0.1530	5.092
TypiClust	40	4	0.2046	5.720
Random	40	4	0.1681	5.799
TypiClust	80	4	0.2577	7.462
Random	80	4	0.2227	6.822

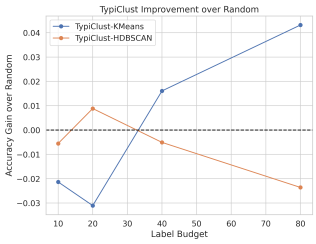
Table 2: Full experimental results across all budgets, seeds, and methods. TypiClust methods dominate over Random selections, over high budgets : This is for the **Unmodified Self-Supervised TPCRP algorithm - Concordant with the papers**[1] **findings, for TypiClust performance relative to Random**

ε	λ	**Clusters**	**Noise (%)**	**Behaviour**
0.4	0.01	60	35%	Very fragmented, high diversity
0.5	0.01	35	20%	Balanced, potentially a good structure
0.6	0.01	20	10%	Larger clusters, diversity penalty important
0.7	0.01	8	5%	Fewer clusters, risk of redundancy

Table 3: Effect of DBSCAN radius ε and diversity penalty λ on cluster structure in the SSL feature space. - **used in the improved algorithm**



(a) Figure 3.1: Visualises the differences in performance across different budgets for TypiClust-HDBSCAN, TypiClust and Random



(b) Figure 3.2: Highlights the total gain the TypiClust-HDBSCAN over 500 epochs received and the relative improvement over various budgets

Method	Budget	Seed	Accuracy	Runtime (s)
TypiClust-KMeans	10	0	0.1137	5.385
TypiClust-HDBSCAN	10	0	0.1404	5.073
Random	10	0	0.1269	5.120
TypiClust-KMeans	20	0	0.1119	5.596
TypiClust-HDBSCAN	20	0	0.1416	5.684
Random	20	0	0.1173	5.668
TypiClust-KMeans	40	0	0.1880	5.365
TypiClust-HDBSCAN	40	0	0.1732	6.422
Random	40	0	0.1638	6.502
TypiClust-KMeans	80	0	0.2603	7.533
TypiClust-HDBSCAN	80	0	0.1919	6.593
Random	80	0	0.2171	7.443
TypiClust-KMeans	10	1	0.1112	5.389
TypiClust-HDBSCAN	10	1	0.1099	4.172
Random	10	1	0.1482	5.318
TypiClust-KMeans	20	1	0.1374	5.740
TypiClust-HDBSCAN	20	1	0.1644	5.733
Random	20	1	0.1689	5.682
TypiClust-KMeans	40	1	0.1983	6.452
TypiClust-HDBSCAN	40	1	0.1844	6.602
Random	40	1	0.1755	6.464
TypiClust-KMeans	80	1	0.2529	6.521
TypiClust-HDBSCAN	80	1	0.1862	6.566
Random	80	1	0.1838	7.558
TypiClust-KMeans	10	2	0.1229	5.251
TypiClust-HDBSCAN	10	2	0.1388	4.106
Random	10	2	0.1476	5.403
TypiClust-KMeans	20	2	0.1162	5.700
TypiClust-HDBSCAN	20	2	0.1504	5.696
Random	20	2	0.1371	4.615
TypiClust-KMeans	40	2	0.1951	6.430
TypiClust-HDBSCAN	40	2	0.1787	6.499
Random	40	2	0.2050	6.530
TypiClust-KMeans	80	2	0.2491	6.492
TypiClust-HDBSCAN	80	2	0.1879	6.623
Random	80	2	0.1987	7.650
TypiClust-KMeans	10	3	0.1115	4.159
TypiClust-HDBSCAN	10	3	0.1256	5.244
Random	10	3	0.1266	5.329
TypiClust-KMeans	20	3	0.0907	5.731
TypiClust-HDBSCAN	20	3	0.1487	5.706
Random	20	3	0.1284	5.698
TypiClust-KMeans	40	3	0.2262	6.442
TypiClust-HDBSCAN	40	3	0.1801	6.500
Random	40	3	0.1973	5.444
TypiClust-KMeans	80	3	0.2413	7.664
TypiClust-HDBSCAN	80	3	0.1812	6.639
Random	80	3	0.2177	7.610
TypiClust-KMeans	10	4	0.1243	4.161
TypiClust-HDBSCAN	10	4	0.1479	5.287
Random	10	4	0.1410	5.280
TypiClust-KMeans	20	4	0.1017	5.704
TypiClust-HDBSCAN	20	4	0.1523	4.648
Random	20	4	0.1615	5.714
TypiClust-KMeans	40	4	0.2048	6.460
TypiClust-HDBSCAN	40	4	0.1902	6.521
Random	40	4	0.1904	5.418
TypiClust-KMeans	80	4	0.2535	7.791
TypiClust-HDBSCAN	80	4	0.1760	6.702
Random	80	4	0.2239	7.607

Table 4: Full experimental results across all budgets, seeds, and methods for the **modified TypiClust algorithm** *Both this and the unmodified were derived from the 'run experiments notebook' from the GitHub repository provided.* **- concordant with Hacohen's paper[1] for TypiClust-Kmeans - this approach successfully matches the paper's approach**

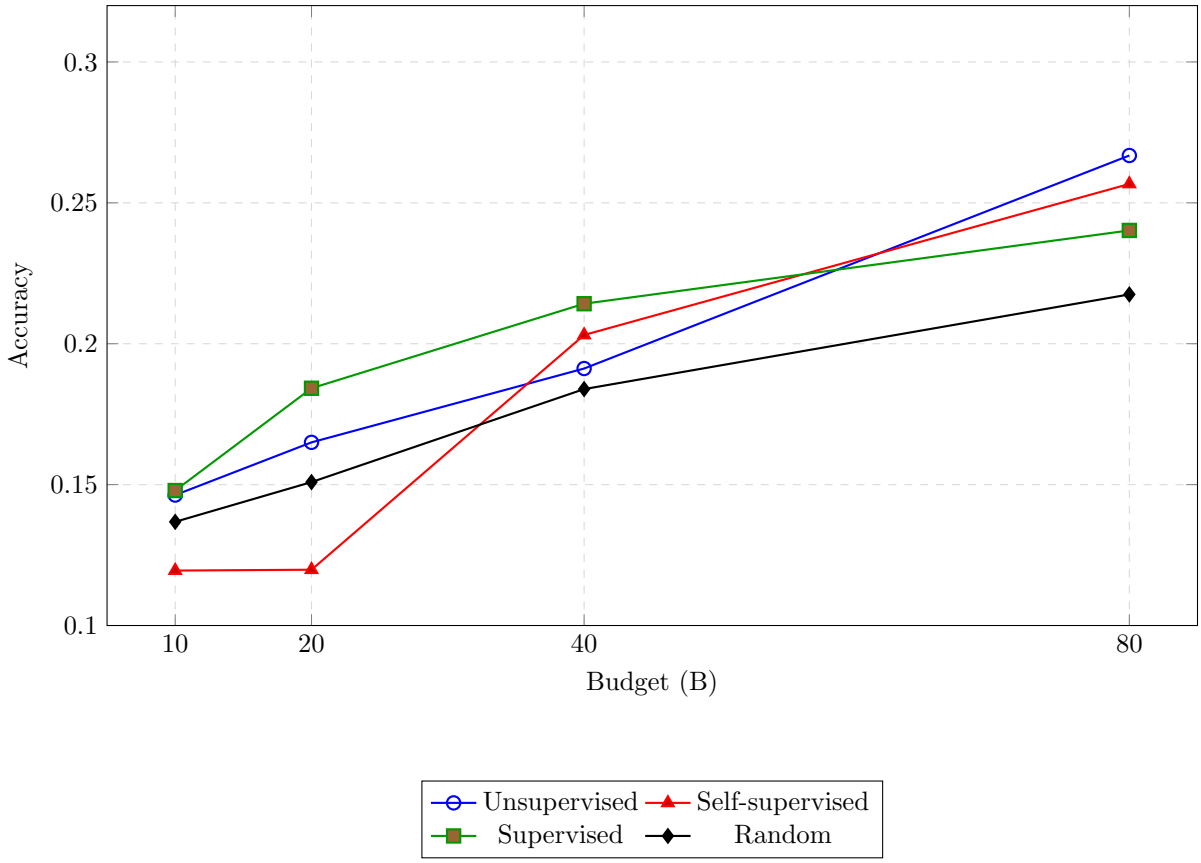
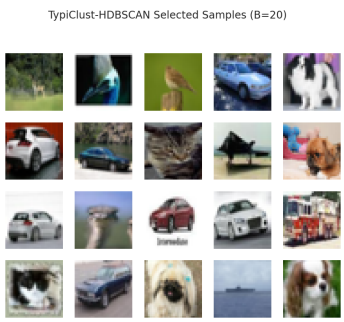
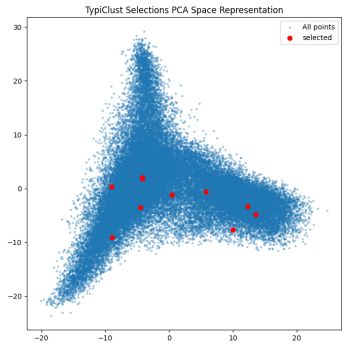


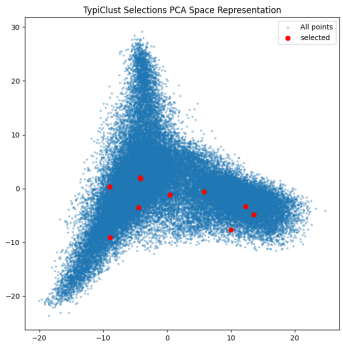
Figure 6: Accuracy vs Budget across **three TypiClust frameworks** and a random baseline. Values are averaged across seeds.



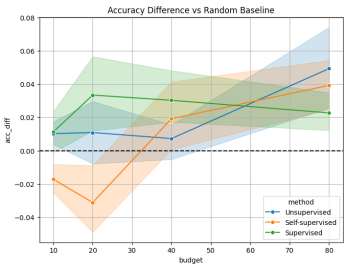
(a) Figure 3.3: Shows the output of the results generated by the TypiClust-HDBSCAN



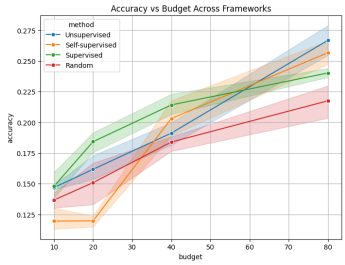
(b) Figure 3.4: A figure demonstrating the distribution of data points across a data manifold considered by the HDBSCAN improved algorithm



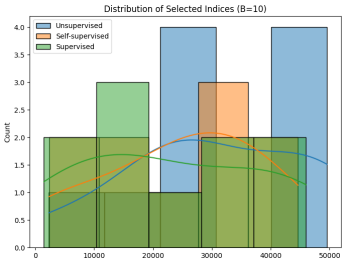
(c) Figure 3.4: A figure demonstrating the distribution of data points across a data manifold considered by the HDBSCAN improved algorithm



(a) Figure 4.1: Comparison of **three TypiClust frameworks**; different across budgets



(b) Figure 4.2: Comparison of **TypiClust frameworks** for accuracy and budget



(c) Figure 4.3: TypiClust selected indices for respective learning frameworks

Budget	TypiClust-KMeans vs Random (p-value)	TypiClust-HDBSCAN vs Random (p-value)
10	6.6504×10^{-3}	9.8117×10^{-1}
20	4.1779×10^{-2}	4.6037×10^{-1}
40	9.0136×10^{-2}	6.4094×10^{-1}
80	1.8545×10^{-3}	2.7570×10^{-1}

Table 5: Paired t-test results comparing each method against the Random baseline across five seeds. Lower p-values indicate stronger evidence that the method outperforms Random.

In [ ]:
```python
import os
import numpy as np

# Torch Import
import torch
import torch.nn as nn
import torch.nn.functional as F


import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Dataset

from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors

from typing import List, Tuple
```

In [ ]:
```python
# Reproducibility
torch.manual_seed(0)
np.random.seed(0)
```

In [ ]:
```python
class ResNetEncd(nn.Module):
    def __init__(self, base="resnet18", proj_dim=128):
        super().__init__()

        backbone = getattr(torchvision.models, base)(pretrained=False)
        self.backbone = nn.Sequential(*list(backbone.children())[:-1])
        feat_dim = backbone.fc.in_features

        self.projection = nn.Sequential(
            nn.Linear(feat_dim, feat_dim),
            nn.ReLU(inplace=True),
            nn.Linear(feat_dim, proj_dim)
        )

    def forward(self, x, return_projection: bool = True):
        feature_val = self.backbone(x)
        feature_val = torch.flatten(feature_val, 1)
        if return_projection:
            proj = self.projection(feature_val)
            return feature_val, proj
        return feature_val

    def represent(self, x):
        return self.forward(x, return_projection=False)


# NT-Xent Loss  - CONSTRASTIVE NTXent
class ContrastiveNTXent(nn.Module):
    def __init__(self, temperature : float = 0.2):
        super().__init__()
        self.temperature = temperature


    def forward(self, z_a: torch.Tensor, z_b : torch.Tensor) -> torch.Tensor:
```

```
        batch = z_a.size(0)
        z = torch.cat([z_a, z_b], dim =0 )
        z = F.normalize(z, dim=1)


        sim_matrix = torch.matmul(z, z.T) / self.temperature

        diag_mask = torch.eye(2 * batch, device= z.device).bool()
        sim_matrix.masked_fill_(diag_mask, -9e15)

        lb_val = torch.arange(batch, device=z.device) # 1D-tensor
        lb_val = torch.cat([lb_val + batch, lb_val], dim=0)

        loss = F.cross_entropy(sim_matrix, lb_val)
        return loss



# VVVVV Augmentation of data Below VVVVV

# 2-Crop Transformation
class TC_Transform:
    def __init__(self):
        self.aug = transforms.Compose([
            transforms.RandomResizedCrop(32, scale=(0.2,1.0)),
            transforms.RandomHorizontalFlip(),
            transforms.ColorJitter(0.4, 0.4, 0.4, 0.1),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
            transforms.Normalize((0.4914, 0.4822, 0.4465),
                                 (0.2470, 0.2435, 0.2616)),
        ])


    def __call__(self, img):
        return self.aug(img), self.aug(img)
```

In [ ]:

```
def comp_tpc(features: np.ndarray, k: int = 20) -> np.ndarray:

    nbrs = NearestNeighbors(n_neighbors=min(k + 1, len(features)), algorithm='au
    distances, _ = nbrs.kneighbors(features)  # distances: [N, k+1]
    distances = distances[:, 1:]
    avg_sq_dist = (distances ** 2).mean(axis=1)
    typicality = 1.0 / (avg_sq_dist + 1e-12)
    return typicality


# Multi-Round Typiclust
def typiclust_selection(
    features: np.ndarray,
    initial_labeled: List[int],
    budget_total: int,
    batch_size_per_round: int,
    k_typicality: int = 20,
    random_state: int = 0,
) -> List[int]:
    N = features.shape[0]
    rng = np.random.RandomState(random_state)

    labeled = set(initial_labeled)
```

```
    unlabeled = set(range(N)) - labeled

    typicality = comp_tpc(features, k=k_typicality)

    while len(labeled) < budget_total:
        current_L = list(labeled)
        current_U = list(unlabeled)

        n_clusters = len(current_L) + batch_size_per_round
        n_clusters = min(n_clusters, len(current_L) + len(current_U))  # safety

        kmeans = KMeans(n_clusters=n_clusters, random_state=rng.randint(1e9))
        cluster_lb_val = kmeans.fit_predict(features)

        cluster_has_label = np.zeros(n_clusters, dtype=bool)
        for idx in current_L:
            c = cluster_lb_val[idx]
            cluster_has_label[c] = True

        uncovered_clusters = [c for c in range(n_clusters) if not cluster_has_la

        cluster_sizes = {c: np.sum(cluster_lb_val == c) for c in uncovered_clust
        sorted_clusters = sorted(uncovered_clusters, key=lambda c: -cluster_size

        new_indices = []
        for c in sorted_clusters:
            if len(labeled) + len(new_indices) >= budget_total:
                break
            cluster_idx = np.where(cluster_lb_val == c)[0]
            cluster_idx = [i for i in cluster_idx if i in unlabeled]
            if not cluster_idx:
                continue
            cluster_typ = typicality[cluster_idx]
            best_local = cluster_idx[int(np.argmax(cluster_typ))]
            new_indices.append(best_local)

        for idx in new_indices:
            labeled.add(idx)
            unlabeled.remove(idx)

        if not new_indices:
            break

    return sorted(labeled)
```

In [ ]:

```
class TwoCropCIFAR_10(Dataset):
    def __init__(self, root: str, train:bool, transform):
        self.dataset = torchvision.datasets.CIFAR10(
            root = root,
            train= train,
            download = True,
            transform = None
        )
        self.transform = transform

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        img, _ = self.dataset[idx]
```

```python
        x1, x2 = self.transform(img)
        return x1, x2, idx



def train_self_supervised(
    encoder : ResNetEncd,
    dataset: Dataset,
    device: torch.device,
    batch_size: int = 256,
    epochs: int = 200,
    lr: float = 1e-3,
    resume_path : str = None
) -> ResNetEncd:
    loader = DataLoader(
        dataset,   # TO DO - ADD DATASET & BATCH_SIZE integration
        batch_size=batch_size,
        shuffle=True,
        num_workers=4,
        drop_last=True
    )

    ssl_losses = []


    encoder = encoder.to(device)
    loss_fn = ContrastiveNTXent(temperature=0.2).to(device)
    optimizer = torch.optim.Adam(encoder.parameters(), lr=lr)


    start_epoch = 0
    if resume_path is not None and os.path.exists(resume_path):
        print(f"Resuming SSL training from checkpoint: {resume_path}")
        checkpoint = torch.load(resume_path, map_location=device)
        encoder.load_state_dict(checkpoint["encoder"])
        optimizer.load_state_dict(checkpoint["optimizer"])
        start_epoch = checkpoint["epoch"] + 1
        print(f"Resumed from epoch {start_epoch}")


    encoder.train()
    for epoch in range(epochs):
        running_loss = 0.0
        for x1, x2, _ in loader:
            x1= x1.to(device)
            x2 = x2.to(device)

            _, z1 = encoder(x1, return_projection=True)

            _, z2 = encoder(x2, return_projection=True)

            loss = loss_fn(z1,z2)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * x1.size(0)

        epoch_loss = running_loss / len(loader.dataset)
        print(f"Epoch {epoch+1}/{epochs} - SSL LOSS : {epoch_loss:.4f}")
```

```
        ssl_losses.append(epoch_loss)

        # Checkpoint every 40 epochs
        if (epoch+1)%40 == 0:
            os.makedirs("budget_results/ssl_checkpoints", exist_ok=True)
            ckpt_path = f"budget_results/ssl_checkpoints/ssl_epoch_{epoch+1}.pth
            torch.save(encoder.state_dict(), ckpt_path)
            print(f"Checkpoint Saved [ CKLP ] at {epoch+1} -> {ckpt_path}")

    np.save("budget_results/ssl_loss.npy", np.array(ssl_losses))
    return encoder




# Extraction of Features
def extract_features(
        encoder : ResNetEncd,
        dataset : torchvision.datasets.CIFAR10,
        device : torch.device,
        batch_size : int = 256
) -> np.ndarray:
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers =4
    )

    encoder.eval()
    all_features = []

    with torch.no_grad():
        for x, _ in loader:
            x = x.to(device)
            features = encoder.represent(x)
            all_features.append(features.cpu().numpy())

    features = np.concatenate(all_features, axis=0)
    return features
```

In [ ]:

```
def run_pipeline_selection(
    data_root: str = "./data",
    budget_total: int = 100,
    batch_size_per_round: int = 10,
    ssl_epochs: int = 200,
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    ssl_transform = TC_Transform()
    ssl_dataset = TwoCropCIFAR_10(root=data_root, train=True, transform=ssl_tran

    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder = train_self_supervised(
        encoder=encoder,
        dataset=ssl_dataset,
        device=device,
        batch_size=256,
        epochs=ssl_epochs,
```

```
        lr=1e-3,
    )

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])
    base_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    features = extract_features(
        encoder=encoder,
        dataset=base_dataset,
        device=device,
        batch_size=256
    )

    #Multi-round TypiClust
    labeled_indices = typiclust_selection(
        features=features,
        initial_labeled=[],
        budget_total=budget_total,
        batch_size_per_round=batch_size_per_round,
        k_typicality=20,
        random_state=0,
    )

    os.makedirs("budget_results", exist_ok=True)
    np.save("budget_results/typiclust_selection.npy", np.array(labeled_indices))
    print(f"Selected {len(labeled_indices)} labeled points (single-round)")

    return labeled_indices




def generate_and_save_typiclust_selections(
        budgets=[10,20,40,80],
        batch_size_per_round=10,
        ssl_epochs=200,
        data_root="./data"

):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    ssl_transform = TC_Transform()
    ssl_dataset = TwoCropCIFAR_10(root=data_root, train=True, transform=ssl_tran

    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder = train_self_supervised(
        encoder=encoder,
        dataset=ssl_dataset,
        device=device,
```

```
        batch_size=256,
        epochs=ssl_epochs,
        lr=1e-3,
    )

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])
    base_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    features = extract_features(
        encoder=encoder,
        dataset=base_dataset,
        device=device,
        batch_size=256
    )

    os.makedirs("budget_results", exist_ok=True)

    for B in budgets:
        labeled_indices = typiclust_selection(
            features=features,
            initial_labeled=[],
            budget_total=B,
            batch_size_per_round=batch_size_per_round,
            k_typicality=20,
            random_state=0,
        )

        np.save(f"budget_results/typiclust_B{B}.npy", np.array(labeled_indices))
        print(f"Saved TypiClust selection for B={B} → budget_results/typiclust_B
```

In [ ]:

```
if __name__ == '__main__':

    #run_pipeline_selection(ssl_epochs=500) # Run pipeline()
    generate_and_save_typiclust_selections(ssl_epochs=500) # reduce due to time
```

In [ ]:
```python
import os
import numpy as np

# Torch Import
import torch
import torch.nn as nn
import torch.nn.functional as F


import torchvision
import torchvision.transforms as transforms

from torch.utils.data import DataLoader, Dataset

from sklearn.neighbors import NearestNeighbors

from typing import List, Tuple

from sklearn.decomposition import PCA

import hdbscan
```

In [ ]:
```python
# Reproducibility
torch.manual_seed(0)
np.random.seed(0)

## MODIFIED
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
## END OF MODIFICATION


SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
os.chdir(SCRIPT_DIR)


class ResNetEncd(nn.Module):
    def __init__(self, base="resnet18", proj_dim=128):
        super().__init__()

        backbone = getattr(torchvision.models, base)(pretrained=False)
        self.backbone = nn.Sequential(*list(backbone.children())[:-1])
        feat_dim = backbone.fc.in_features

        self.projection = nn.Sequential(
            nn.Linear(feat_dim, feat_dim),
            nn.ReLU(inplace=True),
            nn.Linear(feat_dim, proj_dim)
        )

    def forward(self, x, return_projection: bool = True):
        feat = self.backbone(x)
        feat = torch.flatten(feat, 1)
        if return_projection:
            proj = self.projection(feat)
            return feat, proj
        return feat
```

```
    def represent(self, x):
        return self.forward(x, return_projection=False)
```

In [ ]:

```
# Depreciated Use - kept for logging reasons
class ConstructiveNTXent(nn.Module):
    def __init__(self, temperature: float = 0.2):
        super().__init__()
        self.temperature = temperature

    def forward_pass(self, z_a: torch.Tensor, z_b : torch.Tensor) -> torch.Tensc

        batch = z_a.size(0)
        z = torch.cat([z_a, z_b], dim=0)

        sim_matrix = torch.matmul(z, z.T) / self.temperature

        diag_mask = torch.eye(2 * batch ,  device=z.device).bool()

        sim_matrix.masked_fill_(diag_mask, -9e15)

        positives = torch.arange(batch, 2 * batch, device = z.device)
        labels = positives.repeat(2) # Cross-entropy target to be satisfied

        loss = F.cross_entropy(sim_matrix, labels) # Cross-Entropy loss funct.
        return loss


# NT-Xent Loss  - CONSTRASTIVE NTXent
class ContrastiveNTXent(nn.Module):
    def __init__(self, temperature : float = 0.2):
        super().__init__()
        self.temperature = temperature


    def forward(self, z_a: torch.Tensor, z_b : torch.Tensor) -> torch.Tensor:

        batch = z_a.size(0)
        z = torch.cat([z_a, z_b], dim =0 )
        z = F.normalize(z, dim=1)


        sim_matrix = torch.matmul(z, z.T) / self.temperature

        diag_mask = torch.eye(2 * batch, device= z.device).bool()
        sim_matrix.masked_fill_(diag_mask, -9e15)

        labels = torch.arange(batch, device=z.device) # 1D-tensor
        labels = torch.cat([labels + batch, labels], dim=0)

        loss = F.cross_entropy(sim_matrix, labels)
        return loss



# VVVVV Augmentation of data Below VVVVV

# 2-Crop Transformation
class TC_Transform:
    def __init__(self):
        self.aug = transforms.Compose([
```

```
            transforms.RandomResizedCrop(32, scale=(0.2,1.0)),
            transforms.RandomHorizontalFlip(),
            transforms.ColorJitter(0.4, 0.4, 0.4, 0.1),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
            transforms.Normalize((0.4914, 0.4822, 0.4465),
                                 (0.2470, 0.2435, 0.2616)),
        ])


    def __call__(self, img):
        return self.aug(img), self.aug(img)
```

In [ ]:

```
def compute_typicality(features: np.ndarray, k: int = 20) -> np.ndarray:

    nbrs = NearestNeighbors(n_neighbors=min(k + 1, len(features)), algorithm='au
    distances, _ = nbrs.kneighbors(features)  # distances: [N, k+1]
    distances = distances[:, 1:]
    avg_sq_dist = (distances ** 2).mean(axis=1)
    typicality = 1.0 / (avg_sq_dist + 1e-12)
    return typicality


# Multi-Round Typiclust
def typiclust_selection(
    features: np.ndarray,
    initial_labeled: List[int],
    budget_total: int,
    batch_size_per_round: int,
    k_typicality: int = 20,
    random_state: int = 0,
) -> List[int]:


    ###  MODIFIED FROM ORIGINAL ###

    # Modified Distance Parameters
    lambda_ = 0.01

    # DBSCAN Parameters
    #db = DBSCAN(eps=1.2, min_samples=10).fit(features)
    #cluster_labels = db.labels_

    # HDBSCAN
    clusterer = hdbscan.HDBSCAN(
        min_cluster_size = 30,
        min_samples =5,
        metric="euclidean"
    ).fit(features)

    cluster_labels = clusterer.labels_

    unique, counts = np.unique(cluster_labels, return_counts=True)
    n_noise = int(np.sum(cluster_labels == -1))
    n_clusters = int((unique != -1).sum())
    print(f"[HDBSCAN] clusters={n_clusters}, noise={n_noise}, total={len(cluster
    print("Top cluster sizes:", sorted([(c, s) for c, s in zip(unique, counts) i

    # Save metadata for documentation ( HDBSCAN )
    meta = {
```

```
        "min_cluster_size": 30,
        "min_samples": 5,
        "lambda": lambda_,
        "n_clusters": n_clusters,
        "n_noise": n_noise
    }
    os.makedirs("budget_results/meta", exist_ok=True)
    np.save("budget_results/meta/hdbscan_meta.npy", meta)



    ###  END OF MODIFICATION FROM ORIGINAL ###

    N = features.shape[0]
    rng = np.random.RandomState(random_state)

    # Reduction of noise points ( -1 labels)
    valid_mask = cluster_labels != -1
    valid_indices = np.where(valid_mask)[0]

    labeled = set(initial_labeled)
    unlabeled = set(range(N)) - labeled

    typicality = compute_typicality(features, k=k_typicality)

    while len(labeled) < budget_total:


        # DEBUG PRINT
        print(f"[LOOP] labeled = {len(labeled)}, unlabled = {len(unlabeled)}")

        # clusters that DO  [ NOT ]  contain a labeled point
        unique_clusters = np.unique(cluster_labels[valid_mask])

        # DEBUG PRINT
        print(f"[Clusters] unique_clust = {len(unique_clusters)}")

        if unique_clusters.size == 0:
            print("Warning: HDBSCAN found no clusters (all points noise). Increa
            break

        cluster_has_label = {c: False for c in unique_clusters}

        for idx in labeled:
            c = cluster_labels[idx]
            if c != -1:
                cluster_has_label[c] = True

        # DEBUG PRINT
        uncovered_clusters = [c for c in unique_clusters if not cluster_has_labe
        print(f"[Uncovrd] uncovered_clusters = {len(uncovered_clusters)}")

        if len(uncovered_clusters) == 0:
            break

        new_indices = []

        for c in uncovered_clusters:
            if len(labeled) + len(new_indices) >= budget_total:
                break
```

```
            cluster_idx = np.where(cluster_labels == c)[0]
            cluster_idx = [i for i in cluster_idx if i in unlabeled]

            if not cluster_idx:
                continue

            # Diversity penalised scoring
            scores = typicality[cluster_idx] - lambda_ * distance_to_selected(cl
            best_local = cluster_idx[np.argmax(scores)]
            new_indices.append(best_local)

        for idx in new_indices:
            labeled.add(idx)
            unlabeled.remove(idx)

        if not new_indices:
            break



    return sorted(labeled)
    ###  END OF MODIFICATIONS FROM ORIGINAL ###
```

In [ ]:

```
    ###  MODIFIED FROM ORIGINAL ###
def distance_to_selected(indices, features, labeled):
    if len(labeled) == 0:
        return np.zeros(len(indices))

    selected_feats = features[list(labeled)]
    dists = np.linalg.norm(features[indices][:, None] - selected_feats[None, :],
    return dists.min(axis=1)

    ###  END OF MODIFICATIONS FROM ORIGINAL ###



class TwoCropCIFAR_10(Dataset):
    def __init__(self, root: str, train:bool, transform):
        self.dataset = torchvision.datasets.CIFAR10(
            root = root,
            train= train,
            download = True,
            transform = None
        )
        self.transform = transform

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        img, _ = self.dataset[idx]
        x1, x2 = self.transform(img)
        return x1, x2, idx



def train_self_supervised(
    encoder : ResNetEncd,
    dataset: Dataset,
```

```
    device: torch.device,
    batch_size: int = 256,
    epochs: int = 200,
    lr: float = 1e-3,
    resume_path : str = None
) -> ResNetEncd:
    loader = DataLoader(
        dataset,   # TO DO - ADD DATASET & BATCH_SIZE integration
        batch_size=batch_size,
        shuffle=True,
        num_workers=4,
        drop_last=True
    )

    ssl_losses = []


    encoder = encoder.to(device)
    loss_fn = ContrastiveNTXent(temperature=0.2).to(device)
    optimizer = torch.optim.Adam(encoder.parameters(), lr=lr)


    start_epoch = 0
    if resume_path is not None and os.path.exists(resume_path):
        print(f"Resuming SSL training from checkpoint: {resume_path}")
        checkpoint = torch.load(resume_path, map_location=device)
        encoder.load_state_dict(checkpoint["encoder"])
        optimizer.load_state_dict(checkpoint["optimizer"])
        start_epoch = checkpoint["epoch"] + 1
        print(f"Resumed from epoch {start_epoch}")


    encoder.train()
    for epoch in range(epochs):
        running_loss = 0.0
        for x1, x2, _ in loader:
            x1= x1.to(device)
            x2 = x2.to(device)

            _, z1 = encoder(x1, return_projection=True)

            _, z2 = encoder(x2, return_projection=True)

            loss = loss_fn(z1,z2)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * x1.size(0)

        epoch_loss = running_loss / len(loader.dataset)
        print(f"Epoch {epoch+1}/{epochs} - SSL LOSS : {epoch_loss:.4f}")

        ssl_losses.append(epoch_loss)

        # Checkpoint every 40 epochs
        if (epoch+1)%40 == 0:
            os.makedirs("budget_results/ssl_checkpoints", exist_ok=True)
            ckpt_path = f"budget_results/ssl_checkpoints/ssl_epoch_{epoch+1}.pth
            torch.save(encoder.state_dict(), ckpt_path)
```

```
            print(f"Checkpoint Saved [ CKLP ] at {epoch+1} -> {ckpt_path}")

    np.save("budget_results/ssl_loss.npy", np.array(ssl_losses))
    return encoder
```

In [ ]:

```
# Extraction of Features
def extract_features(
        encoder : ResNetEncd,
        dataset : torchvision.datasets.CIFAR10,
        device : torch.device,
        batch_size : int = 256
) -> np.ndarray:
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers =4
    )

    encoder.eval()
    all_features = []

    with torch.no_grad():
        for x, _ in loader:
            x = x.to(device)
            features = encoder.represent(x)
            all_features.append(features.cpu().numpy())

    features = np.concatenate(all_features, axis=0)
    return features



def run_pipeline_selection(
    data_root: str = "./data",
    budget_total: int = 100,
    batch_size_per_round: int = 10,
    ssl_epochs: int = 200,
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    ssl_transform = TC_Transform()
    ssl_dataset = TwoCropCIFAR_10(root=data_root, train=True, transform=ssl_tran

    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder = train_self_supervised(
        encoder=encoder,
        dataset=ssl_dataset,
        device=device,
        batch_size=256,
        epochs=ssl_epochs,
        lr=1e-3,
    )

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
```

```
    ])
    base_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    features = extract_features(
        encoder=encoder,
        dataset=base_dataset,
        device=device,
        batch_size=256
    )

    # Reduce dimensionality for easier computation
    pca = PCA(n_components=50)
    features = pca.fit_transform(features)

    #Multi-round TypiClust
    labeled_indices = typiclust_selection(
        features=features,
        initial_labeled=[],
        budget_total=budget_total,
        batch_size_per_round=batch_size_per_round,
        k_typicality=20,
        random_state=0,
    )

    os.makedirs("budget_results", exist_ok=True)
    np.save("budget_results/typiclust_selection.npy", np.array(labeled_indices))
    print(f"Selected {len(labeled_indices)} labeled points (single-round)")

    return labeled_indices
```

In [ ]:

```
def generate_and_save_typiclust_selections(
        budgets=[10,20,40,80],
        batch_size_per_round=10,
        ssl_epochs=500,
        data_root="./data"

):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    ssl_transform = TC_Transform()
    ssl_dataset = TwoCropCIFAR_10(root=data_root, train=True, transform=ssl_tran

    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder = train_self_supervised(
        encoder=encoder,
        dataset=ssl_dataset,
        device=device,
        batch_size=256,
        epochs=ssl_epochs,
        lr=1e-3,
    )

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
```

```
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])
    base_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    features = extract_features(
        encoder=encoder,
        dataset=base_dataset,
        device=device,
        batch_size=256
    )

    os.makedirs("budget_results", exist_ok=True)

    for B in budgets:
        labeled_indices = typiclust_selection(
            features=features,
            initial_labeled=[],
            budget_total=B,
            batch_size_per_round=batch_size_per_round,
            k_typicality=20,
            random_state=0,
        )

        np.save(f"budget_results/typiclust_B{B}.npy", np.array(labeled_indices))
        print(f"Saved TypiClust selection for B={B} → budget_results/typiclust_B


def execute_HDBSCAN_PROCESS(
        features_path="modified_budget_results/features.npy",
        checkpoint_path="modified_budget_results/ssl_checkpoints/ssl_epoch_480.p
        budgets=[10,20,40,80],
        lambda_=0.01
):




    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Load your trained encoder
    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder.load_state_dict(torch.load(checkpoint_path, map_location=device))
    encoder = encoder.to(device)
    encoder.eval()

    # Load extracted features
    features = np.load(features_path)

    pca = PCA(n_components=50)
    features = pca.fit_transform(features)

    results_dir = "modified_budget_results/hdbscan_results"
    os.makedirs(results_dir, exist_ok=True)
```

```
    for B in budgets:
        labeled_indices = typiclust_selection(
            features=features,
            initial_labeled=[],
            budget_total=B,
            batch_size_per_round=10,
            k_typicality=20,
            random_state=0,
        )

        out_path = f"{results_dir}/typiclust_HDBSCAN_B{B}.npy"
        np.save(out_path, np.array(labeled_indices))

        print(f"HDBSCAN selection saved for B={B} → {out_path}")
```

In [ ]:

```
if __name__ == '__main__':
    #generate_and_save_typiclust_selections(budgets=[10], ssl_epochs=5) # For te
    run_pipeline_selection(ssl_epochs=500) # Run pipeline()


    # To generate 500 epochs & perform DBSCAN
    #generate_and_save_typiclust_selections(ssl_epochs=500) # reduce due to time

    # To perform strictly only DBSCAN
    execute_HDBSCAN_PROCESS(
        features_path="modified_budget_results/features.npy",
        checkpoint_path="modified_budget_results/ssl_checkpoints/ssl_epoch_480.p
        budgets=[10,20,40,80],
        #eps=200.0, # No longer needed for HDBSCAN
        lambda_=0.01
    )
```

In [ ]:
```python
import os
import numpy as np
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Dataset
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from typing import List
```

In [ ]:
```python
torch.manual_seed(0)
np.random.seed(0)
```

In [ ]:
```python
class ResNetEncd(nn.Module):
    def __init__(self, base="resnet18", proj_dim=128):
        super().__init__()
        # Pretrained = True to extract powerful offline representations without
        import warnings
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            # Handle newer PyTorch API where pretrained is deprecated
            try:
                backbone = getattr(torchvision.models, base)(weights=torchvision
            except AttributeError:
                backbone = getattr(torchvision.models, base)(pretrained=True)

        self.backbone = nn.Sequential(*list(backbone.children())[:-1])
        feat_dim = backbone.fc.in_features
        self.projection = nn.Sequential(
            nn.Linear(feat_dim, feat_dim),
            nn.ReLU(inplace=True),
            nn.Linear(feat_dim, proj_dim)
        )

    def forward(self, x, return_projection: bool = True):
        feat = self.backbone(x)
        feat = torch.flatten(feat, 1)
        if return_projection:
            proj = self.projection(feat)
            return feat, proj
        return feat

    def represent(self, x):
        return self.forward(x, return_projection=False)


def compute_typicality(features: np.ndarray, k: int = 20) -> np.ndarray:
    nbrs = NearestNeighbors(n_neighbors=min(k + 1, len(features)), algorithm='au
    distances, _ = nbrs.kneighbors(features)  # distances: [N, k+1]
    distances = distances[:, 1:]
    avg_sq_dist = (distances ** 2).mean(axis=1)
    typicality = 1.0 / (avg_sq_dist + 1e-12)
    return typicality

def typiclust_selection(
    features: np.ndarray,
    initial_labeled: List[int],
```

```
    budget_total: int,
    batch_size_per_round: int,
    k_typicality: int = 20,
    random_state: int = 0,
) -> List[int]:
    N = features.shape[0]
    rng = np.random.RandomState(random_state)

    labeled = set(initial_labeled)
    unlabeled = set(range(N)) - labeled

    typicality = compute_typicality(features, k=k_typicality)

    while len(labeled) < budget_total:
        current_L = list(labeled)
        current_U = list(unlabeled)

        n_clusters = len(current_L) + batch_size_per_round
        n_clusters = min(n_clusters, len(current_L) + len(current_U))  # safety

        kmeans = KMeans(n_clusters=n_clusters, random_state=rng.randint(1e9))
        cluster_labels = kmeans.fit_predict(features)

        cluster_has_label = np.zeros(n_clusters, dtype=bool)
        for idx in current_L:
            c = cluster_labels[idx]
            cluster_has_label[c] = True

        uncovered_clusters = [c for c in range(n_clusters) if not cluster_has_la

        cluster_sizes = {c: np.sum(cluster_labels == c) for c in uncovered_clust
        sorted_clusters = sorted(uncovered_clusters, key=lambda c: -cluster_size

        new_indices = []
        for c in sorted_clusters:
            if len(labeled) + len(new_indices) >= budget_total:
                break
            cluster_idx = np.where(cluster_labels == c)[0]
            cluster_idx = [i for i in cluster_idx if i in unlabeled]
            if not cluster_idx:
                continue
            cluster_typ = typicality[cluster_idx]
            best_local = cluster_idx[int(np.argmax(cluster_typ))]
            new_indices.append(best_local)

        for idx in new_indices:
            labeled.add(idx)
            unlabeled.remove(idx)

        if not new_indices:
            break

    return sorted(labeled)
```

In [ ]:

```
def extract_features(
    encoder: ResNetEncd,
    dataset: torchvision.datasets.CIFAR10,
    device: torch.device,
    batch_size: int = 256
) -> np.ndarray:
```

```python
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=4
    )

    encoder.eval()
    all_features = []

    with torch.no_grad():
        for x, _ in loader:
            x = x.to(device)
            features = encoder.represent(x)
            all_features.append(features.cpu().numpy())

    features = np.concatenate(all_features, axis=0)
    return features


def run_pipeline_unsupervised(
    data_root: str = "./data",
    budget_total: int = 100,
    batch_size_per_round: int = 10,
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])

    base_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    # Use untrained representations OR imageNet representations.
    encoder = ResNetEncd(base="resnet18", proj_dim=128).to(device)

    features = extract_features(
        encoder=encoder,
        dataset=base_dataset,
        device=device,
        batch_size=256
    )

    labeled_indices = typiclust_selection(
        features=features,
        initial_labeled=[],
        budget_total=budget_total,
        batch_size_per_round=batch_size_per_round,
        k_typicality=20,
        random_state=0,
    )
```

```
    os.makedirs("unsupervised_budget_results", exist_ok=True)
    np.save("unsupervised_budget_results/typiclust_unsupervised.npy", np.array(l
    print(f"Selected {len(labeled_indices)} labeled points (unsupervised)")

    return labeled_indices
```

In [ ]:

```
def generate_and_save_typiclust_selections(
        budgets=[10, 20, 40, 80],
        batch_size_per_round=10,
        data_root="./data"
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])

    base_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    # Use untrained representations OR imageNet representations.
    encoder = ResNetEncd(base="resnet18", proj_dim=128).to(device)

    features = extract_features(
        encoder=encoder,
        dataset=base_dataset,
        device=device,
        batch_size=256
    )

    os.makedirs("unsupervised_budget_results", exist_ok=True)

    for B in budgets:
        labeled_indices = typiclust_selection(
            features=features,
            initial_labeled=[],
            budget_total=B,
            batch_size_per_round=batch_size_per_round,
            k_typicality=20,
            random_state=0,
        )

        np.save(f"unsupervised_budget_results/typiclust_B{B}.npy", np.array(labe
        print(f"Saved TypiClust selection for B={B} → unsupervised_budget_result
```

In [ ]:

```
if __name__ == '__main__':
    # run_pipeline_unsupervised()
    generate_and_save_typiclust_selections()
```

In [ ]:
```python
import os
import numpy as np
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Dataset
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from typing import List
```

In [ ]:
```python
torch.manual_seed(0)
np.random.seed(0)
```

In [ ]:
```python
class ResNetEncd(nn.Module):
    def __init__(self, base="resnet18", proj_dim=128):
        super().__init__()
        backbone = getattr(torchvision.models, base)(pretrained=False)
        self.backbone = nn.Sequential(*list(backbone.children())[:-1])
        feat_dim = backbone.fc.in_features
        self.projection = nn.Sequential(
            nn.Linear(feat_dim, feat_dim),
            nn.ReLU(inplace=True),
            nn.Linear(feat_dim, proj_dim)
        )

    def forward(self, x, return_projection: bool = True):
        feat = self.backbone(x)
        feat = torch.flatten(feat, 1)
        if return_projection:
            proj = self.projection(feat)
            return feat, proj
        return feat

    def represent(self, x):
        return self.forward(x, return_projection=False)
```

In [ ]:
```python
def compute_typicality(features: np.ndarray, k: int = 20) -> np.ndarray:
    nbrs = NearestNeighbors(n_neighbors=min(k + 1, len(features)), algorithm='au
    distances, _ = nbrs.kneighbors(features)  # distances: [N, k+1]
    distances = distances[:, 1:]
    avg_sq_dist = (distances ** 2).mean(axis=1)
    typicality = 1.0 / (avg_sq_dist + 1e-12)
    return typicality

def typiclust_selection(
    features: np.ndarray,
    initial_labeled: List[int],
    budget_total: int,
    batch_size_per_round: int,
    k_typicality: int = 20,
    random_state: int = 0,
) -> List[int]:
    N = features.shape[0]
    rng = np.random.RandomState(random_state)

    labeled = set(initial_labeled)
```

```
    unlabeled = set(range(N)) - labeled

    typicality = compute_typicality(features, k=k_typicality)

    while len(labeled) < budget_total:
        current_L = list(labeled)
        current_U = list(unlabeled)

        n_clusters = len(current_L) + batch_size_per_round
        n_clusters = min(n_clusters, len(current_L) + len(current_U))  # safety

        kmeans = KMeans(n_clusters=n_clusters, random_state=rng.randint(1e9))
        cluster_labels = kmeans.fit_predict(features)

        cluster_has_label = np.zeros(n_clusters, dtype=bool)
        for idx in current_L:
            c = cluster_labels[idx]
            cluster_has_label[c] = True

        uncovered_clusters = [c for c in range(n_clusters) if not cluster_has_la

        cluster_sizes = {c: np.sum(cluster_labels == c) for c in uncovered_clust
        sorted_clusters = sorted(uncovered_clusters, key=lambda c: -cluster_size

        new_indices = []
        for c in sorted_clusters:
            if len(labeled) + len(new_indices) >= budget_total:
                break
            cluster_idx = np.where(cluster_labels == c)[0]
            cluster_idx = [i for i in cluster_idx if i in unlabeled]
            if not cluster_idx:
                continue
            cluster_typ = typicality[cluster_idx]
            best_local = cluster_idx[int(np.argmax(cluster_typ))]
            new_indices.append(best_local)

        for idx in new_indices:
            labeled.add(idx)
            unlabeled.remove(idx)

        if not new_indices:
            break

    return sorted(labeled)
```

In [ ]:

```
def train_supervised(
    encoder: ResNetEncd,
    dataset: Dataset,
    device: torch.device,
    batch_size: int = 256,
    epochs: int = 500,
    lr: float = 1e-3,
    num_classes: int = 10,
    resume_path: str = None
) -> ResNetEncd:
    # Train the encoder using traditional supervision on labels
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=True,
```

```
        num_workers=4,
        drop_last=True
    )

    classifier = nn.Linear(128, num_classes).to(device)
    encoder = encoder.to(device)

    loss_fn = nn.CrossEntropyLoss().to(device)
    optimizer = torch.optim.Adam(list(encoder.parameters()) + list(classifier.pa

    start_epoch = 0
    if resume_path is not None and os.path.exists(resume_path):
        print(f"Resuming Supervised training from checkpoint: {resume_path}")
        checkpoint = torch.load(resume_path, map_location=device)
        encoder.load_state_dict(checkpoint["encoder"])
        classifier.load_state_dict(checkpoint["classifier"])
        optimizer.load_state_dict(checkpoint["optimizer"])
        start_epoch = checkpoint["epoch"] + 1
        print(f"Resumed from epoch {start_epoch}")

    encoder.train()
    classifier.train()

    sup_losses = []

    for epoch in range(start_epoch, epochs):
        running_loss = 0.0
        for x, y in loader:
            x = x.to(device)
            y = y.to(device)

            _, z = encoder(x, return_projection=True)
            preds = classifier(z)

            loss = loss_fn(preds, y)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * x.size(0)

        epoch_loss = running_loss / len(loader.dataset)
        print(f"Epoch {epoch+1}/{epochs} - Supervised LOSS : {epoch_loss:.4f}")
        sup_losses.append(epoch_loss)

        if (epoch+1) % 40 == 0:
            os.makedirs("supervised_budget_results/sup_checkpoints", exist_ok=Tr
            ckpt_path = f"supervised_budget_results/sup_checkpoints/sup_epoch_{e
            torch.save({
                "encoder": encoder.state_dict(),
                "classifier": classifier.state_dict(),
                "optimizer": optimizer.state_dict(),
                "epoch": epoch
            }, ckpt_path)
            print(f"Checkpoint Saved at {epoch+1} -> {ckpt_path}")

    os.makedirs("supervised_budget_results", exist_ok=True)
    np.save("supervised_budget_results/sup_loss.npy", np.array(sup_losses))
    return encoder
```

In [ ]:

```python
def extract_features(
    encoder: ResNetEncd,
    dataset: torchvision.datasets.CIFAR10,
    device: torch.device,
    batch_size: int = 256
) -> np.ndarray:
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=4
    )

    encoder.eval()
    all_features = []

    with torch.no_grad():
        for x, _ in loader:
            x = x.to(device)
            features = encoder.represent(x)
            all_features.append(features.cpu().numpy())

    features = np.concatenate(all_features, axis=0)
    return features


def run_pipeline_supervised(
    data_root: str = "./data",
    budget_total: int = 100,
    batch_size_per_round: int = 10,
    sup_epochs: int = 500,
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])

    sup_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder = train_supervised(
        encoder=encoder,
        dataset=sup_dataset,
        device=device,
        batch_size=256,
        epochs=sup_epochs,
        lr=1e-3,
    )
```

```
    features = extract_features(
        encoder=encoder,
        dataset=sup_dataset,
        device=device,
        batch_size=256
    )

    labeled_indices = typiclust_selection(
        features=features,
        initial_labeled=[],
        budget_total=budget_total,
        batch_size_per_round=batch_size_per_round,
        k_typicality=20,
        random_state=0,
    )

    os.makedirs("supervised_budget_results", exist_ok=True)
    np.save("supervised_budget_results/typiclust_supervised.npy", np.array(label
    print(f"Selected {len(labeled_indices)} labeled points (supervised)")

    return labeled_indices

def generate_and_save_typiclust_selections(
        budgets=[10, 20, 40, 80],
        batch_size_per_round=10,
        sup_epochs=500,
        data_root="./data"
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])

    sup_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder = train_supervised(
        encoder=encoder,
        dataset=sup_dataset,
        device=device,
        batch_size=256,
        epochs=sup_epochs,
        lr=1e-3,
    )

    features = extract_features(
        encoder=encoder,
        dataset=sup_dataset,
        device=device,
        batch_size=256
```

```
    )

    os.makedirs("supervised_budget_results", exist_ok=True)

    for B in budgets:
        labeled_indices = typiclust_selection(
            features=features,
            initial_labeled=[],
            budget_total=B,
            batch_size_per_round=batch_size_per_round,
            k_typicality=20,
            random_state=0,
        )

        np.save(f"supervised_budget_results/typiclust_B{B}.npy", np.array(labele
        print(f"Saved TypiClust selection for B={B} → supervised_budget_results/
```

In [ ]:

```
if __name__ == '__main__':
    # run_pipeline_supervised(sup_epochs=500)
    generate_and_save_typiclust_selections(sup_epochs=500)
```